Erişilebilirlik

Flutter'ın erişilebilirlik desteği hakkında bilgi.

Arka plan

Uygulamaların geniş bir kullanıcı yelpazesi için erişilebilir olmasını sağlamak, yüksek kaliteli bir uygulama geliştirmenin temel bir parçasıdır. Kötü tasarlanmış uygulamalar, her yaştan insan için engeller oluşturur. Birleşmiş Milletler Engelli Kişilerin Haklarına İlişkin Sözleşmesi, bilgi sistemlerine evrensel erişimi sağlamanın ahlaki ve yasal zorunluluğunu belirtir; dünyanın dört bir yanındaki ülkeler erişilebilirliği bir gereklilik olarak uygular; ve şirketler, hizmetlerine erişimi en üst düzeye çıkarmanın ticari avantajlarını kabul eder.

Uygulamanızı yayınlamadan önce erişilebilirlik kontrol listesini temel bir kriter olarak eklemenizi şiddetle tavsiye ederiz. Flutter, geliştiricilerin uygulamalarını daha erişilebilir hale getirmelerine destek olmaya kararlıdır ve temel işletim sisteminin sağladığına ek olarak, erişilebilirlik için birinci sınıf çerçeve desteği içerir, bunlar şunlardır:

Erişilebilirlik düzenlemeleri

Erişilebilirlik standartları ve düzenlemeleri, ürünlerin engelli bireyler için erişilebilir olmasını sağlamaya yardımcı olur. Bunların çoğu yasa ve politikalara dönüştürülerek ürün ve hizmetler için zorunlu hale getirilmiştir.

- **WCAG 2** : Web İçeriği Erişilebilirlik Yönergeleri (WCAG) 2, web içeriğini engelli kişiler için daha erişilebilir hale getirmek amacıyla uluslararası alanda tanınan bir standarttır. Dünya Çapında Web Konsorsiyumu (W3C) tarafından geliştirilen istikrarlı, teknik bir standarttır.
- **EN 301 549** : EN 301 549 , Bilgi ve İletişim Teknolojisi (BİT) ürün ve hizmetleri için erişilebilirlik gereksinimlerine ilişkin Avrupa uyumlu standardıdır.
- **VPAT** : Gönüllü Ürün Erişilebilirlik Şablonu (VPAT) , erişilebilirlik gereksinimlerini ve standartlarını ürün ve hizmetler için uygulanabilir test kriterlerine dönüştüren ücretsiz bir şablondur.

Dünya genelindeki yasalar, dijital içerik ve hizmetlerin engelli bireyler için erişilebilir olmasını gerektirmektedir. ABD'de, Engelli Amerikalılar Yasası (ADA), kamuya açık yerlerde ayrımcılığı yasaklamaktadır. Rehabilitasyon Yasası'nın 508. maddesi, federal kurumların ve yüklenicilerinin tüm bilgi ve iletişim teknolojileri (BİT) için WCAG standartlarını karşılamasını gerektirmektedir.

AB'de, Avrupa Erişilebilirlik Yasası (EAA), temel olarak EN 301 549 standardını kullanarak, çok çeşitli kamu ve özel sektör hizmetlerinin erişilebilir olmasını şart koşmaktadır

Erişilebilirliği göz önünde bulundurarak inşa etmek

Uygulamanızın herkes tarafından kullanılabilmesini sağlamak, erişilebilirliği en başından itibaren uygulamaya entegre etmek anlamına gelir. Bazı uygulamalar için bu, söylendiği kadar kolay değildir. Aşağıdaki videoda, iki mühendisimiz, erişilebilirlik açısından son derece kötü durumda olan bir mobil uygulamayı, Flutter'ın yerleşik widget'larından yararlanarak çok daha erişilebilir bir deneyim sunan bir uygulamaya dönüştürüyor.

Erişilebilirlik yayın kontrol listesi

İşte uygulamanızı yayınlamaya hazırlarken göz önünde bulundurmanız gereken hususların eksiksiz olmayan bir listesi.

- **Aktif etkileşimler** . Tüm aktif etkileşimlerin bir şey yapmasını sağlayın. Basılabilen her düğme basıldığında bir şey yapmalıdır. Örneğin, bir olay için hiçbir şey yapmayan bir geri çağırma işleviniz varsa , bunu ekranda hangi kontrole bastığınızı açıklayan `onPressed`bir mesaj gösterecek şekilde değiştirin .`SnackBar`
- **Ekran okuyucu testi** . Ekran okuyucu, üzerine dokunduğunuzda sayfadaki tüm kontrolleri tanımlayabilmeli ve açıklamalar anlaşılır olmalıdır. Uygulamanızı TalkBack (Android) ve VoiceOver (iOS) ile test edin.
- **Kontrast oranları** . Devre dışı bırakılmış bileşenler hariç, kontroller veya metin ile arka plan arasında en az 4,5:1 kontrast oranına sahip olmanızı öneririz. Görsellerin de yeterli kontrasta sahip olup olmadığı kontrol edilmelidir.
- **Bağlam değiştirme** . Kullanıcı bilgi yazarken hiçbir şey kullanıcının bağlamını otomatik olarak değiştirmemelidir. Genel olarak, widget'lar bir tür onay işlemi olmadan kullanıcının bağlamını değiştirmekten kaçınmalıdır.
- **Dokunulabilir hedefler** . Tüm dokunulabilir hedeflerin en az 48x48 piksel olması gerekir.
- **Hatalar** . Önemli işlemler geri alınabilir olmalıdır. Hata gösteren alanlarda, mümkünse bir düzeltme önerin.
- **Renk görme bozukluğu testi** . Kontroller, renk körlüğü ve gri tonlama modlarında kullanılabilir ve okunabilir olmalıdır.
- **Ölçek faktörleri** . Kullanıcı arayüzü, metin boyutu ve ekran ölçeklendirmesi için çok büyük ölçek faktörlerinde bile okunabilir ve kullanılabilir kalmalıdır.

Kullanıcı arayüzü tasarımı ve stillendirmesi

Erişilebilir bir uygulama oluşturmak için, kullanıcı arayüzünüzü erişilebilirliği göz önünde bulundurarak tasarlayın. Bu sayfa, erişilebilir kullanıcı arayüzü tasarımı ve stilinin temel yönlerini kapsamaktadır.

Büyük yazı tipleri

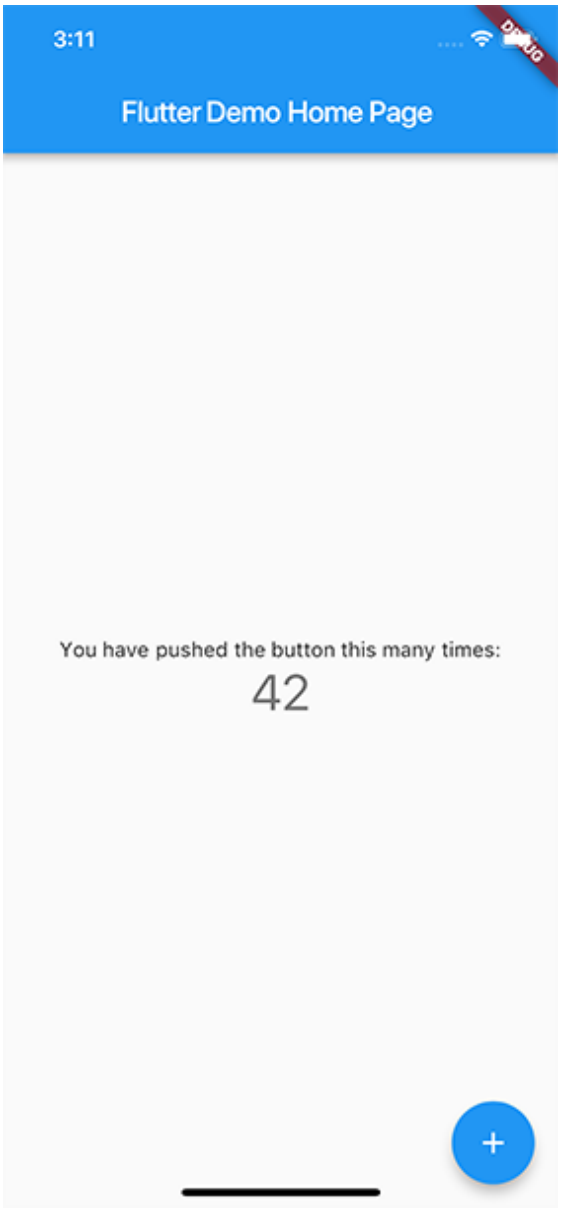
Hem Android hem de iOS, uygulamalar tarafından kullanılan istenen yazı tipi boyutlarını yapılandırmak için sistem ayarları içerir. Flutter metin bileşenleri, yazı tipi boyutlarını belirlerken bu işletim sistemi ayarını dikkate alır.

Flutter, yazı tipi boyutlarını işletim sistemi ayarlarına göre otomatik olarak hesaplar. Ancak, bir geliştirici olarak, yazı tipi boyutları arttırıldığında düzeninizin tüm içeriğini görüntülemek için yeterli alana sahip olduğundan emin olmalısınız. Örneğin, uygulamanızın tüm bölümlerini en büyük yazı tipi ayarını kullanacak şekilde yapılandırılmış küçük ekranlı bir cihazda test edebilirsiniz.

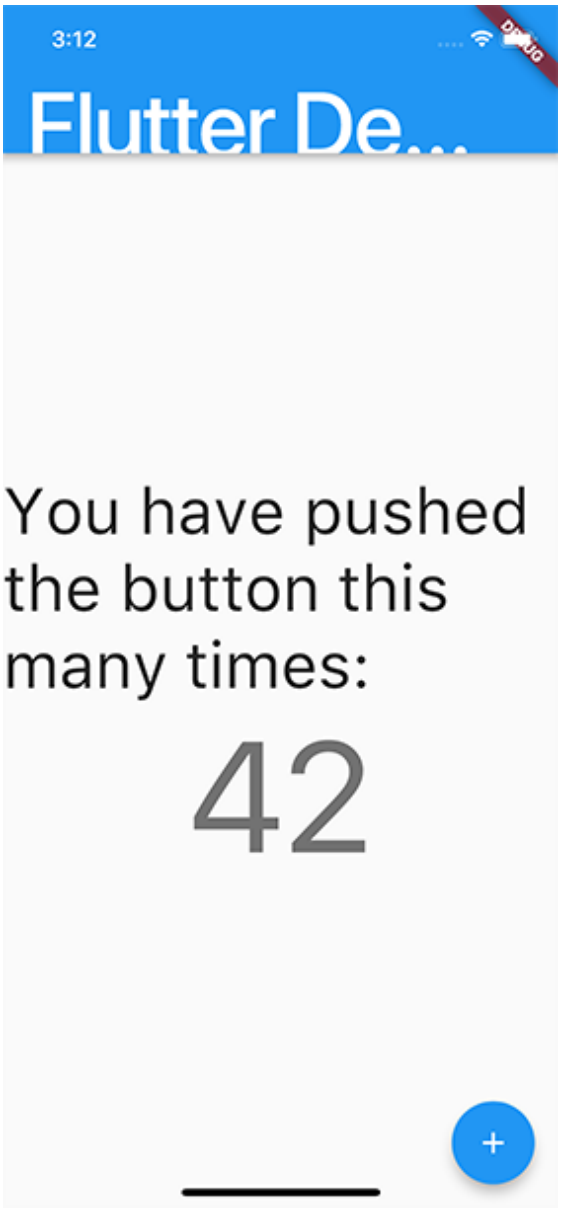
Yazı tipi boyutunu ayarlamak için: iOS'ta Ayarlar > Erişilebilirlik > Ekran ve Metin Boyutu'na gidin; Android'de Ayarlar > Yazı tipi boyutu'na gidin.

Örnek

Aşağıdaki iki ekran görüntüsü, varsayılan iOS yazı tipi ayarıyla ve iOS erişilebilirlik ayarlarında en büyük yazı tipi ayarı seçilerek oluşturulmuş standart Flutter uygulama şablonunu göstermektedir.



Varsayılan yazı tipi ayarı



En büyük erişilebilirlik yazı tipi ayarı

Yeterli kontrast

Yeterli renk kontrastı, metin ve görsellerin okunmasını kolaylaştırır. Çeşitli görme engeli olan kullanıcılara fayda sağlamasının yanı sıra, yeterli renk kontrastı, doğrudan güneş ışığına maruz kalma veya düşük parlaklığa sahip bir ekran gibi aşırı aydınlatma koşullarında cihazlardaki bir arayüzü görüntülerken tüm kullanıcılara yardımcı olur.

W3C şunları önermektedir :

- Küçük yazı tipleri için (18 punto normal veya 14 punto kalın yazı tipinden küçük) en az 4,5:1 oranında en boy oranı gereklidir.
- Büyük yazı tipleri için en az 3.0:1 (18 punto ve üzeri normal veya 14 punto ve üzeri kalın)

Flutter'ın Erişilebilirlik Yönergeleri API'sini kullanarak kontrastı test edebilirsiniz . Test etme hakkında daha fazla bilgi için erişilebilirlik testi sayfasına göz atın .

Dokunma hedef boyutu

Çok küçük olan kontroller birçok kişi için etkileşim kurmayı ve seçim yapmayı zorlaştırır. Etkileşimli öğelerin, kullanıcılar tarafından kolayca basılabilmesi için yeterince büyük bir dokunma alanına sahip olduğundan emin olun.

Hem Android hem de iOS, sırasıyla minimum 48x48 dp ve 44x44 pts dokunma hedefi boyutu önermektedir.

W3C , minimum hedef boyutunun 44 x 44 CSS piksel olmasını önermektedir.

Flutter'ın Erişilebilirlik Yönergeleri API'sini kullanarak dokunma hedefi boyutunu test edebilirsiniz . Test etme hakkında daha fazla bilgi için erişilebilirlik testi sayfasına göz atın .

Diğer erişilebilirlik özellikleri

Platform tarafından etkinleştirilebilecek ek erişilebilirlik özelliklerini (örneğin kalın metin, yüksek kontrast ve ters renkler) AccessibilityFeatures sınıfında inceleyebilirsiniz .

Erişilebilirlik teknolojileri

Özet

Yardımcı teknolojiler, dijital içeriği engelli bireyler için erişilebilir hale getirmede hayati öneme sahiptir. Bu belge, Flutter geliştirme ile ilgili iki temel yardımcı teknoloji kategorisine genel bir bakış sunmaktadır: görme engelli kullanıcılar için ekran okuyucular ve motor kısıtlılığı olanlar için hareket destek araçları. Bu teknolojileri anlayarak ve test ederek, Flutter uygulamanızın herkes için daha kapsayıcı ve kullanıcı dostu bir deneyim sağlamasını sağlayabilirsiniz.

Ekran okuyucular

Mobil cihazlarda, ekran okuyucular (TalkBack , VoiceOver), görme engelli kullanıcıların ekran içeriği hakkında sesli geri bildirim almalarını ve mobil cihazlarda jestler, masaüstü cihazlarda ise klavye kısayolları kullanarak kullanıcı arayüzüyle etkileşim kurmalarını sağlar. Mobil cihazınızda VoiceOver veya TalkBack'i açın ve uygulamanızda gezinin.

Uygulamanızda belirli bir sesle seslendirilmesi gereken metinler varsa, `.` yöntemini çağırarak ekran okuyucuya hangi sesi kullanacağını bildirin TextSpan.locale. `MaterialApp.locale` Bu yöntem, `Localizations.override` Flutter 3.38 sürümünden itibaren ekran okuyucu seslerini etkileyecektir. Genellikle, ekran okuyucu, `.` yöntemiyle açıkça belirtmediğiniz durumlar dışında sistem sesini kullanır `TextSpan.locale`.

Hareketlilik desteği

El becerisi veya el gücü sınırlı olan kullanıcılar için, hareketlilik destek özellikleri faydalı olabilir. Hem Android hem de iOS, gezinmeyi ve kontrolü kolaylaştırmak için tasarlanmış bir dizi araç sunar. Bu özellikler, kullanıcıların cihazlarını harici anahtarlar, sesli komutlar veya basitleştirilmiş ekran menüleri aracılığıyla çalıştırmalarına olanak tanır.

Android, Switch Access, Voice Access ve Accessibility Menu özelliklerini sunarken, iOS Switch Control, Voice Control ve AssistiveTouch özelliklerini sunmaktadır. Bu araçları anlamak, farklı fiziksel yeteneklere sahip kişiler tarafından kullanılabilir uygulamalar oluşturmaya yardımcı olur.

İşletim sistemi	Özellikler	Fonksiyonlar
Android	**Anahtar Erişimi**	Alternatif bir giriş yöntemi olarak, Anahtar Erişimi ve Kamera Anahtarlarını kullanabilirsiniz.
Android	**Sesli Erişim**	Cihazınızı sesinizle kontrol edin.
Android	**Erişilebilirlik Menüsü**	Ekran üzerinde beliren, basitleştirilmiş düğmelerle telefonun temel işlevlerini kontrol etmeyi sağlayan yüzer bir menü.
iOS	**Anahtar Kontrolü**	Alternatif giriş yöntemleri olarak anahtarları kullanın.
iOS	**Ses Kontrolü**	Cihazınızı sesinizle kontrol edin.
iOS	**Yardımcı Dokunma**	Çoklu parmak hareketlerini veya donanım düğmesi işlemlerini AssistiveTouch ile değiştirebilirsiniz.

Erişilebilirlik testi

Erişilebilirlik düzenlemeleri

Uygulamanızın erişilebilir olduğundan emin olmak için, Web İçerik Erişilebilirlik Yönergeleri (WCAG) 2 , EN 301 549 gibi kamu standartlarına göre kontrol edin ve ürününüzü kendi kendinize değerlendirmek için Gönüllü Ürün Erişilebilirlik Şablonu (VPAT) gibi kaynaklardan yararlanın. Bu düzenlemeler hakkında daha fazla bilgi için ana erişilebilirlik sayfasına göz atın .

Erişilebilirlik desteğinin incelenmesi
Aşağıdakileri test etmek için otomatik erişilebilirlik tarayıcıları kullanmanızı öneririz:

- Android için:
    - Android için Erişilebilirlik Tarayıcısını Yükleyin
    - **Android Ayarları > Erişilebilirlik > Erişilebilirlik Tarayıcısı > Açık seçeneğinden** Erişilebilirlik Tarayıcısını etkinleştirin .
    - Tarama işlemini başlatmak için Erişilebilirlik Tarayıcısı 'onay kutusu' simge düğmesine gidin.
- iOS için:
    - `iOS`Xcode'da Flutter uygulamanızın klasörünü açın .
    - Hedef olarak bir Simülatör seçin ve **Çalıştır** düğmesine tıklayın.
    - Xcode'da, **Xcode > Geliştirici Araçlarını Aç > Erişilebilirlik Denetleyicisi'ni** seçin .
    - Erişilebilirlik Denetleyicisinde, **Denetim > Denetim Noktasını Etkinleştir'i** seçin ve ardından çalışan Flutter uygulamanızdaki çeşitli kullanıcı arayüzü öğelerini seçerek erişilebilirlik özelliklerini inceleyin.
    - Erişilebilirlik Denetleyicisinde, araç çubuğundan **Denetim'i** seçin ve ardından olası sorunların raporunu almak için **Denetimi Çalıştır'ı seçin.**
- Web için:
    - Chrome Geliştirici Araçlarını (veya diğer tarayıcılardaki benzer araçları) açın.
    - Flutter tarafından oluşturulan ARIA özniteliklerini içeren semantics host altındaki HTML ağacını inceleyin.
    - Chrome'da, "Öğeler" sekmesinde, anlam ağacına aktarılan verileri incelemek için kullanılabilecek bir "Erişilebilirlik" alt sekmesi bulunur.

Mobil cihazlarda erişilebilirliğin test edilmesi

Uygulamanızı Flutter'ın Erişilebilirlik Yönergeleri API'sini kullanarak test edin . Bu API, uygulamanızın kullanıcı arayüzünün Flutter'ın erişilebilirlik önerilerine uygun olup olmadığını kontrol eder. Bu öneriler metin kontrastı, hedef boyutu ve hedef etiketleri için geçerlidir.

Aşağıdaki kod parçası, Guideline API'nin "" adlı örnek bir widget üzerinde nasıl kullanılacağını göstermektedir `AccessibleApp`:

```
import 'package:flutter_test/flutter_test.dart';

import 'package:your_accessible_app/main.dart';


void main() {

  testWidgets('Follows a11y guidelines', (tester) async {

    final SemanticsHandle handle = tester.ensureSemantics();

    await tester.pumpWidget(const AccessibleApp());


    // Checks that tappable nodes have a minimum size of 48 by 48 pixels

    // for Android.

    await expectLater(tester, meetsGuideline(androidTapTargetGuideline));


    // Checks that tappable nodes have a minimum size of 44 by 44 pixels

    // for iOS.

    await expectLater(tester, meetsGuideline(iOSTapTargetGuideline));


    // Checks that touch targets with a tap or long press action are labeled.

    await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
```

```
    // Checks whether semantic nodes meet the minimum text contrast levels.
    // The recommended text contrast is 3:1 for larger text
    // (18 point and above regular).
    await expectLater(tester, meetsGuideline(textContrastGuideline));
    handle.dispose();
  });
}
```

Bu testleri denemek için, ile oluşturulmuş yenif lutter create bir uygulamada çalıştırın Bu uygulamanın ana ekranındaki her düğme, 18 punto yazı tipinde metin içeren dokunulabilir bir hedef görevi görür.

Guideline API testlerini diğer widget testlerinin `test/a11y_test.dart` yanına veya bu örnekte olduğu gibi ayrı bir dosyaya ekleyebilirsiniz .

Web'de erişilebilirliğin test edilmesi

Profil ve yayın modlarında aşağıdaki komut satırı bayrağını kullanarak web uygulamanız için oluşturulan anlamsal düğümleri görselleştirerek erişilebilirliği hata ayıklayabilirsiniz:

```
flutter run -d chrome --profile --dart-define=FLUTTER_WEB_DEBUG_SHOW_SEMANTICS=true
```

Bu işaret etkinleştirildiğinde, anlamsal düğümler widget'ların üzerinde görünür; anlamsal öğelerin olması gereken yerde olup olmadığını doğrulayabilirsiniz. Anlamsal düğümler yanlış yerleştirilmişse, lütfen bir hata raporu gönderin .

Web erişilebilirliği

Arka plan

Flutter, dahili Semantik ağacını ekran okuyucuların anlayabileceği erişilebilir bir HTML DOM yapısına çevirerek web erişilebilirliğini destekler. Flutter, kullanıcı arayüzünü tek bir tuval üzerinde oluşturduğu için, arayüzün anlamını ve yapısını web tarayıcılarına göstermek için özel bir katmana ihtiyaç duyar.

İsteğe bağlı web erişilebilirliği

Görünmez düğme

Performans nedenleriyle, Flutter'ın web erişilebilirliği varsayılan olarak açık değildir. Erişilebilirliği açmak için kullanıcının görünmez bir düğmeye basması gerekir `aria-label="Enable accessibility"`. Düğmeye basıldıktan sonra, DOM ağacı tüm widget'lar için erişilebilirlik bilgilerini yansıtacaktır.

Kodda erişilebilirlik modunu etkinleştirin

Alternatif bir yaklaşım ise, uygulamayı çalıştırırken aşağıdaki kodu ekleyerek erişilebilirlik modunu etkinleştirmektir.

```
import 'package:flutter/semantics.dart';


void main() {
  runApp(const MyApp());
  if (kIsWeb) {
    SemanticsBinding.instance.ensureSemantics();
  }
}
```

Semantik Rollerle Erişilebilirliği Geliştirme

Semantik roller nelerdir?

Semantik roller, bir kullanıcı arayüzü öğesinin amacını tanımlayarak ekran okuyucuların ve diğer yardımcı araçların uygulamanızı kullanıcılara etkili bir şekilde yorumlamasına ve sunmasına yardımcı olur. Örneğin, bir rol, bir widget'ın düğme mi, bağlantı mı, başlık mı, kaydırıcı mı yoksa tablonun bir parçası mı olduğunu kullanıcılara gösterebilir.

Flutter'ın standart bileşenleri genellikle bu anlamları otomatik olarak sağlarken, açıkça tanımlanmış bir rolü olmayan özel bir bileşen, ekran okuyucu kullanan bir kullanıcı için anlaşılmaz olabilir.

Uygun rolleri atayarak şunları sağlarsınız:

- Ekran okuyucular, öğelerin türünü ve amacını doğru bir şekilde duyurabilir.

- Kullanıcılar, yardımcı teknolojileri kullanarak uygulamanızda daha etkili bir şekilde gezinebilirler.
- Uygulamanız web erişilebilirlik standartlarına uygun olup, kullanılabilirliği artırmaktadır.

`SemanticsRole` Flutter'da web için kullanım

Flutter, geliştiricilerin widget'larına belirli roller atamasına olanak tanıyan bir enum özelliği sunar . SemanticsFlutter web uygulamanız oluşturulduğunda, bu Flutter'a özgü roller, web sayfasının HTML yapısında karşılık gelen ARIA rollerine çevrilir. SemanticsRole

1. Standart Widget'lardan Otomatik Anlam Çıkarma

`TabBar`<img>`, `MenuAnchor<img>` ve ` <img> ` gibi birçok standart Flutter bileşeni, `Table`rollerinin yanı sıra otomatik olarak anlamsal bilgiler de içerir. Mümkün olduğunca, erişilebilirlik yönlerinin çoğunu otomatik olarak ele aldıkları için bu standart bileşenleri kullanmayı tercih edin.

2. Rolleri açıkça eklemek veya geçersiz kılmak

Özel bileşenler için veya varsayılan anlamlar yeterli olmadığında, `Semantics`rolü tanımlamak için widget'ı kullanın:

İşte bir listeyi ve öğelerini açıkça nasıl tanımlayabileceğinize dair bir örnek:

```
import 'package:material_ui/material_ui.dart';

import 'package:flutter/semantics.dart';



class MyCustomListWidget extends StatelessWidget {

  const MyCustomListWidget({Key? key}) : super(key: key);


  @override

  Widget build(BuildContext context) {
```

```
    // This example shows how to explicitly assign list and listitem roles
    // when building a custom list structure.
    return Semantics(
      role: SemanticsRole.list,
      explicitChildNodes: true,
      child: Column(
        children: <Widget>[
          Semantics(
            role: SemanticsRole.listItem,
            child: const Padding(
              padding: EdgeInsets.all(8.0),
              child: Text('Content of the first custom list item.'),
            ),
          ),
          Semantics(
            role: SemanticsRole.listItem,
            child: const Padding(
              padding: EdgeInsets.all(8.0),
              child: Text('Content of the second custom list item.'),
            ),
          ),
        ],
      ),
    );
  }
}
```